Lecția 1 — Primele 20 de puncte

Andrei · Teorie · ferestre, autentificare, validare, date trimise mai departe · subiectul OJTI 2025, cerințele 1 și 2

1. Ce se notează

La județeană primești un subiect cu 4–5 cerințe, un folder de resurse (imagini, pictograme, fișiere `.txt`) și 4 ore. Evaluatorul nu citește codul: **pornește aplicația și bifează baremul**. Azi faci primele două cerințe:

Din baremul OJTI 2025	Puncte
Cerința 1 — formularul Autentificare (controale, date de acces, mesaj + ștergere, trecere la Exploratori, design)	10
Cerința 2 — formularul Exploratori (pictogramă + imagine + casetă + buton, mesaj de eroare + ștergere, butonul Set, design)	10

În barem apare de fiecare dată „**conform cerinței**”. Un titlu de fereastră scris altfel decât în enunț sau un mesaj reformulat pierde punctul, chiar dacă aplicația merge.

2. Proiectul și forma de pornire

Șablonul este **Windows Forms App (.NET Framework)**, pe 4.7.2. Prima fereastră care apare e cea din `Program.cs`:

```
Application.Run(new Autentificare());
```

Atenție la diferența dintre **numele clasei** (`Autentificare`, apare în cod) și **proprietatea** `Text` (titlul de pe bară, pe care îl vede evaluatorul). Baremul verifică titlul. Redenumești `Form1` din Solution Explorer: Visual Studio întreabă dacă redenumește și clasa; răspunzi **Yes**.

> **Capcana: `.slnx`.** Visual Studio 2022 nou salvează soluțiile ca `.slnx`. La concurs ai Visual Studio 2017/2019, care **nu deschide** `.slnx`: evaluatorul nu-ți vede proiectul. Lucrezi în `Lectia02.sln` din repo și nu creezi altă soluție.

3. Trecerea la fereastra următoare: `Show` + `Hide`

Trecerea are două părți: deschizi fereastra nouă, apoi o faci să dispară pe cea curentă.

```
Echipaj echipaj = new Echipaj();
echipaj.Show();
Hide();
```

- `Show()` deschide fereastra și codul merge mai departe imediat.
- `Hide()` o ascunde pe cea curentă: obiectul trăiește mai departe, doar nu se mai vede.

> **De știut de azi.** Dacă închizi din X o fereastră de după autentificare, `Autentificare` e tot pornită, doar ascunsă: butonul roșu Stop din Visual Studio rămâne aprins. Azi oprești aplicația cu Stop. Cum faci ca X-ul să oprească tot e prima temă din lecția următoare.

4. Imagini și pictograme: doar din Designer

Cum	Unde ajunge imaginea	Portabil?
Din Designer: `Image` → **Import…** (pe `PictureBox`, pe `Button`) sau `Icon` pe formă	copiată în `.resx`-ul formei și **încorporată în `.exe`**	✅ da
Din cod: `Image.FromFile(@"C:\Users\...\barca.png")`	citită de pe disc, la fiecare pornire	❌ pe calculatorul evaluatorului calea nu există → aplicația crapă

Regula de azi: **nicio cale către vreun fișier în cod**. Toate imaginile exemplului sunt puse din Designer.

Pictograma de pe bara de titlu = proprietatea `Icon` a formei. Primește doar fișiere `.ico` (`barca.ico` din resurse), nu `.png`.

5. Butonul care este o imagine

„Un buton de acces, reprezentat printr-o imagine”: pui un `Button`, iar imaginea o pui din Designer (`Image` → Import). Restul proprietăților, tot din Properties:

Proprietatea	Valoarea	De ce
`Text`	(gol)	se vede doar imaginea
`FlatStyle`	`Flat`	fără aspectul de buton gri
`FlatAppearance → BorderSize`	`0`	fără ramă
`Cursor`	`Hand`	mâna arată că se poate apăsa

`AcceptButton` este o proprietate a **formei**, nu a butonului: butonul pe care îl apasă tasta Enter, oriunde ar fi cursorul.



Exemplul: butonul-imagine, parola cu `*`, Enter = Start

6. Parola cu `*`

Pui în Properties `PasswordChar = *` pe caseta parolei. Textul real rămâne în `txtParola.Text`; doar afișarea se schimbă.

7. Mesajele

```
MessageBox.Show("Date de acces gresite, mai incearca!", "Conectare",
    MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

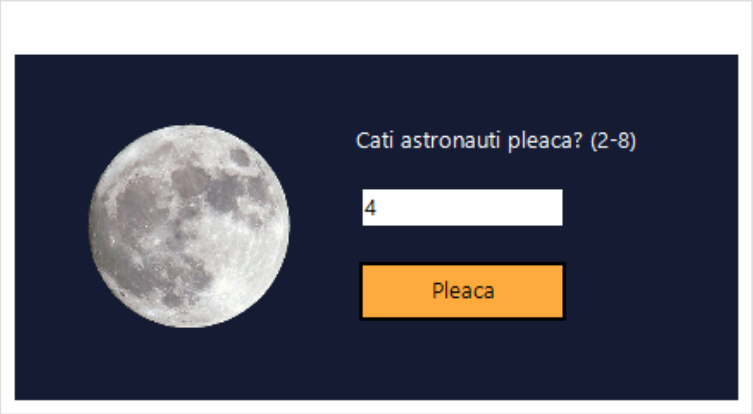
- „mesaj de **avertizare**” → `MessageBoxIcon.Warning`; „mesaj de **eroare**” → `MessageBoxIcon.Error`. Evaluatorul vede iconița.
- Textul îl **copiezi din enunț**, nu îl reformulezi.
- `MessageBox.Show` oprește codul până apasă utilizatorul OK. Abia apoi rulează `Clear()`-urile de după el: „mesaj, apoi ștergere”, exact ordinea din enunț.

8. Validarea unui număr într-un interval

```
int astronauti;
bool esteNumar = int.TryParse(txtAstronauti.Text, out astronauti);
if (!esteNumar || astronauti < 2 || astronauti > 8)
{
    MessageBox.Show(...);
    txtAstronauti.Clear();
    txtAstronauti.Focus();
    return;
}
```

Un singur `if` prinde toate cazurile: text, gol, zecimal, negativ, prea mic, prea mare. Dacă `esteNumar` e `false`, `||` nu mai evaluează restul. Iar `return` oprește trecerea mai departe: fără el, mesajul apare și fereastra următoare se deschide oricum.

> **Capcana: `int.Parse`.** `int.Parse("patru")` nu întoarce nimic: aruncă `FormatException` și aplicația crapă în fața evaluatorului. La fel caseta goală. `int.TryParse` nu aruncă niciodată: îți spune prin `bool` dacă a reușit.



Exemplul: o imagine, o casetă, un buton. Verificat cu 1, 9, abc, gol, 2,5, -3: toate refuzate

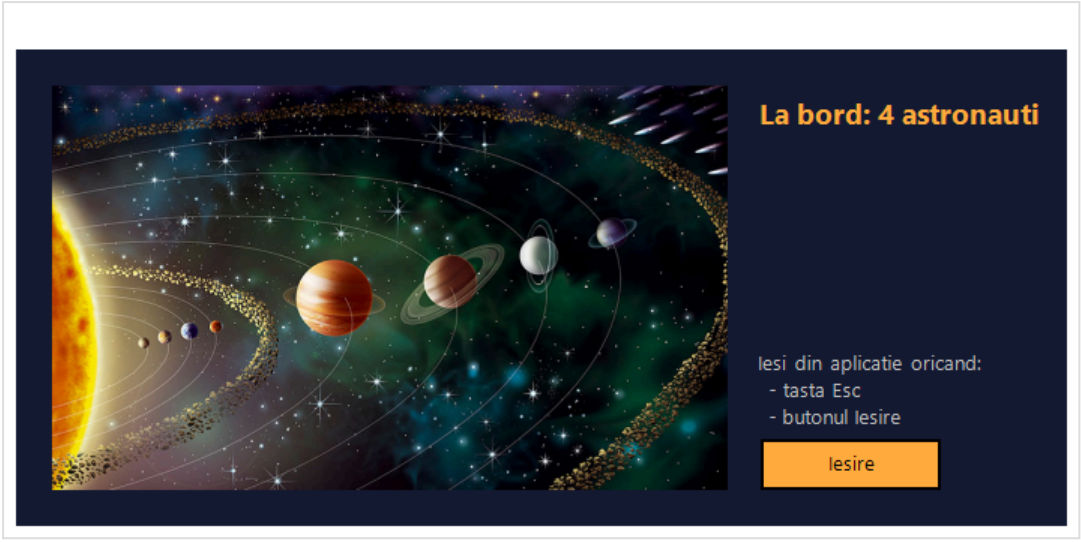
9. Datele merg mai departe prin constructor

Numărul validat în Exploratori trebuie să ajungă în Expeditie. Fereastra nouă îl primește prin constructor, ca orice obiect:

```
public PanouMisiune(int astronauti)
{
    InitializeComponent();
    lblEchipaj.Text = "La bord: " + astronauti + " astronauti";
}
```

```
PanouMisiune panou = new PanouMisiune(astronauti);
```

`InitializeComponent()` rămâne **prima** linie: abia după ea există `lblEchipaj`.



Exemplul: numărul primit prin constructor, pe eticheta de sus

Exemplul din repo

`exemplu/` (proiectul `ExempluMisiune`, în `Lectia02.sln`) aplică tot ce e mai sus, pe resursele de la OJTI 2024: Conectare (`comandant@misiune.ro` / `Luna2024`) → Echipaj (2–8 astronauți) → Misiune. Are deja și ieșirea cu Esc și cu butonul: pe aceea o înveți în lecția următoare.